

DEFINITIONS

DATA STRUCTURE AND ALGORITHMS

Raya Llyanne B. Palmes - BSIS01

Introduction

Data structures and algorithms, these both are fundamental concepts in programming and which are interconnected. For, data structures are methods of organizing and storing data, while on the other hand, algorithms are step-by-step procedures used to solve problems or perform tasks.

To make programs more organized and efficient, they work together. Data structures determine how an information is stored and accessed, while algorithms will determine how that information is processed. Understanding both of them helps programmers develop programs that can solve problems efficiently and manage data effectively (Kosinski, 2025).

Linear Data Structure

For a linear data structure, it is a type of data structure in which elements are arranged in a sequential or the word itself "linear" order. Each of the elements is connected to the next element, to allow data to be processed one after another.

This structure organizes data in a simple sequence, making it useful for storing and processing information in an ordered manner, which is linear. Common examples include arrays, linked lists, stacks, and queues. Arrays store elements in consecutive positions, linked lists connect elements through links, stacks follow Last In, First Out (LIFO), and queues follow First In, First Out (FIFO) (Kosinski, 2025).

One-Dimension Array

A one-dimensional array called the '1D array' is a data structure that stores a collection of elements of the same data type in a single, ordered sequence. Each element is identified and accessed using an index, which usually starts at 0.

A 1D array is organized in a single row or list of elements, giving a convenient access to data through its index. It is commonly used to store lists of values such as student grades, prices, names, or scores. Its advantages include a simple organization, a fast access to elements, and efficient memory use. However, its size is generally fixed after creation, making it less flexible when the number of elements changes (BillWagner, n.d.).

The Big O Notation

Big O notation is a method used in describing how the time or resources required by an algorithm grow as the input size increases. This helps programmers analyze and compare the efficiency of algorithms without depending on the specific computer being used (CS312 Lecture 11: Big-O Notation; Red-black Trees, n.d.).

Big O notation describes the growth rate of an algorithm. Common complexities include:

Complexity	Simple Example	Meaning
O(1)	Accessing an array element by index	Constant time
O(log n)	Binary search	Input is repeatedly divided
O(n)	Linear search	Checks elements one by one
O(n log n)	Merge sort	Divides and processes the data efficiently
O(n²)	Nested loops	Compares or processes elements in pairs

As the input becomes larger, algorithms with slower-growing complexities generally require fewer operations relative to algorithms with faster-growing complexities.

Basic Searching Algorithms

Proceeding to searching algorithms there are methods used to look or find a specific element or value within a collection of data. Searching Algorithms helps programs locate information efficiently and determine whether a particular value exists in a data set. Two common searching algorithms are linear search and binary search.

Linear Search

Linear search checks each element one by one until the target value is found or all elements have been checked. It has a time complexity of $O(n)$ in the worst case, meaning the number of comparisons can increase as the number of elements increases. It is simple to implement and does not require the data to be sorted. However, it can be slower for large collections because it may need to check every element. It is appropriate for small or unsorted lists.

Binary Search

Binary search works on a sorted array by checking the middle element and then repeats dividing the search area in half. It is comparing the target with the middle element and then continues searching in either the left or right half. Binary search has a worst-case time complexity of $O(\log n)$, in reverse of linear and requires the data to be sorted before searching. Its main advantage is that it can find elements much faster than linear search when working with large, sorted collections. However, it cannot be directly used on unsorted data without sorting it first.

REFERENCES

- Kosinski, G. L. M. (2026b, May 14). *What is a data structure?*
<https://www.ibm.com/think/topics/data-structure?>
- BillWagner. (n.d.). *The array reference type - C# reference*. Microsoft Learn.
<https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/builtin-types/arrays?>
- CS312 Lecture 11: *Big-O notation; Red-black trees*. (n.d.).
<https://www.cs.cornell.edu/courses/cs312/2002fa/lectures/lec11.htm?>
- Linear Search (With code)*. (n.d.-b). <https://www.programiz.com/dsa/linear-search?>
- Binary Search (With Code)*. (n.d.). <https://www.programiz.com/dsa/binary-search?>